

tf.compat.v1.nn.sigmoid_cross_entropy_with_logits

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/sigmoid_cross_entropy_with_logits

Computes sigmoid cross entropy given logits.

Function Signatures

```
tf.compat.v1.nn.sigmoid_cross_entropy_with_logits(  
    labels=None, logits=None, name=None )  
  
z * -log(sigmoid(x)) + (1 - z) * -log(1 - sigmoid(x)) = z * -  
log(1 / (1 + exp(-x))) + (1 - z) * -log(exp(-x) / (1 + exp(-  
x))) = z * log(1 + exp(-x)) + (1 - z) * (-log(exp(-x)) +  
log(1 + exp(-x))) = z * log(1 + exp(-x)) + (1 - z) * (x +  
log(1 + exp(-x))) = (1 - z) * x + log(1 + exp(-x)) = x - x * z  
+ log(1 + exp(-x))
```

Code Examples

```
tf.compat.v1.nn.sigmoid_cross_entropy_with_logits(  
    labels=None, logits=None, name=None  
)
```

```
tf.compat.v1.nn.sigmoid_cross_entropy_with_logits(  
labels=None, logits=None, name=None  
)
```

```
z * -log(sigmoid(x)) + (1 - z) * -log(1 - sigmoid(x))  
= z * -log(1 / (1 + exp(-x))) + (1 - z) * -log(exp(-x) / (1 + exp(-  
x)))  
= z * log(1 + exp(-x)) + (1 - z) * (-log(exp(-x)) + log(1 + exp(-  
x)))  
= z * log(1 + exp(-x)) + (1 - z) * (x + log(1 + exp(-x)))  
= (1 - z) * x + log(1 + exp(-x))  
= x - x * z + log(1 + exp(-x))
```

```
z * -log(sigmoid(x)) + (1 - z) * -log(1 - sigmoid(x))  
= z * -log(1 / (1 + exp(-x))) + (1 - z) * -log(exp(-x) / (1 + exp(-  
x)))  
= z * log(1 + exp(-x)) + (1 - z) * (-log(exp(-x)) + log(1 + exp(-  
x)))  
= z * log(1 + exp(-x)) + (1 - z) * (x + log(1 + exp(-x)))  
= (1 - z) * x + log(1 + exp(-x))  
= x - x * z + log(1 + exp(-x))
```

```
x - x * z + log(1 + exp(-x))  
= log(exp(x)) - x * z + log(1 + exp(-x))  
= - x * z + log(1 + exp(x))
```

```
x - x * z + log(1 + exp(-x))  
= log(exp(x)) - x * z + log(1 + exp(-x))  
= - x * z + log(1 + exp(x))
```

```
max(x, 0) - x * z + log(1 + exp(-abs(x)))
```

```
max(x, 0) - x * z + log(1 + exp(-abs(x)))
```

Documentation

Computes sigmoid cross entropy given logits.

Measures the probability error in tasks with two outcomes in which each outcome is independent and need not have a fully certain label. For instance, one could perform a regression where the probability of an event happening is known and used as a label. This loss may also be used for binary classification, where labels are either zero or one.

For brevity, let x = logits, z = labels. The logistic loss is

For $x < 0$, to avoid overflow in $\exp(-x)$, we reformulate the above

Hence, to ensure stability and avoid overflow, the implementation uses this equivalent formulation

logits and labels must have the same type and shape.

Compared to the losses which handle multiple outcomes, `tf.nn.softmax_cross_entropy_with_logits` for general multi-class classification and `tf.nn.sparse_softmax_cross_entropy_with_logits` for more efficient multi-class classification with hard labels, `sigmoid_cross_entropy_with_logits` is a slight simplification for binary classification:

$$\frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + e^0}$$

While `sigmoid_cross_entropy_with_logits` works for soft binary labels (probabilities between 0 and 1), it can also be used for binary classification where the labels are hard. There is an equivalence between all three symbols in this case, with a probability 0 indicating the second class or 1 indicating the first class:

Returns

Raises